Лабораторна робота №12

Семенюка Владислава Григоровича

Група ІПЗ-21

Варіант 7

Код консольної програми

```
using System.Linq;
using Lab12;

class Program
{
    static void Main()
    {
        const string divider =
"-------------------------------------------------
------------";
        const int variant = 7;
        const int limit = 2 + variant % 4;
        const string type = "G";


        Console.WriteLine(divider);
        Console.WriteLine("Lab №12");
        Console.WriteLine("Fullname: Vlad Semeniuk");
        Console.WriteLine("Group: IPZ-21");
        Console.WriteLine($"Variant: {variant}");
        Console.WriteLine($"Limit: {limit}");
        Console.WriteLine(divider);


        // Initialize the default values
        List<BuildingEntry> buildings =
        [
```

```
            new BuildingEntry(code:
$"{type}-{variant}-1", address: "Потебні 5",
floorCount: 8),
            new BuildingEntry(code:
$"{type}-{variant}-2", address: "Львівська 75",
floorCount: 6),
            new BuildingEntry(code:
$"{type}-{variant}-3", address: "Ковельська 10",
floorCount: 4),
            new BuildingEntry(code:
$"{type}-{variant}-4", address: "Лесі Українки 45",
floorCount: 2),
            new BuildingEntry(code:
$"{type}-{variant}-5", address: "Рівенська 33",
floorCount: 7),
            new BuildingEntry(code:
$"{type}-{variant}-6", address: "Набережна 15",
floorCount: 7),
            new BuildingEntry(code:
$"{type}-{variant}-7", address: "Відродження 25",
floorCount: 5),
            new BuildingEntry(code:
$"{type}-{variant}-8", address: "Дубнівська 5",
floorCount: 10)
        ];

        Console.WriteLine("Starting values:\n");

        // Print out the default values
        foreach (var building in buildings)
        {
            Console.WriteLine(building.GetInfo());
        }

        // Filter the default values
```

```
        IEnumerable<string> query =
buildings.Where(building => building.FloorCount >=
limit)
            .OrderByDescending(building =>
building.FloorCount).
            ThenBy(building => building.Address).
            Select(building => $"{building.Code} |
{building.Address} | {building.FloorCount} floor");

        // Print out the values after filtering
        Console.WriteLine("\nThe values after
filtering:\n");
        foreach (var building in query)
        {
            Console.WriteLine(building);
        }

        Console.WriteLine(divider);


      Console.WriteLine($"\nAmount of building that
have >= than 5 is: " +
                        $"{buildings.Count(building
=> building.FloorCount >= limit)}");
      Console.WriteLine($"The biggest floor is:
{buildings.Max(building => building.FloorCount)}");


      // Check if a building exists in a list
      Console.WriteLine("Find a building by
code:\n");
      string[] testCodes =
      {
          $"{buildings[0].Code}",
          $"{type}-{variant}-10"
      };
```

```
      foreach (string testCode in testCodes)
      {

Console.WriteLine(buildings.FirstOrDefault(building
=> building.Code == testCode) == null
              ? "The building is not found"
              : $"The Building exists: {testCode}");
      }

      Console.WriteLine(divider);
      Console.WriteLine("Querying buildings the in
the second method:\n");

      // Filtering using the second method
      IEnumerable<string> secondQuery = from
building in buildings where building.FloorCount >=
limit
          orderby building.FloorCount
              descending, building.Address
          select $"{building.Code} |
{building.Address} | {building.FloorCount} floor";

      // Print out every filtered building
      foreach (var building in secondQuery)
      {
          Console.WriteLine(building);
      }

      // Check if filtered lists are the same
      Console.WriteLine($"\nIs two queries are the
same: {query.SequenceEqual(secondQuery)}");

      Console.WriteLine(divider);

      List<int> numbers = new List<int> { 1, 2, 3 };
      IEnumerable<int> deferred =
numbers.Where(number => number >= 2);
```

```
      numbers.Add(4);
      Console.WriteLine("Query after adding 4: " +
string.Join(", ", deferred));
      List<int> saved = deferred.ToList();
      numbers.Add(5);
      Console.WriteLine("Saved list: " +
string.Join(", ", saved));
      Console.WriteLine("Query after adding 5: " +
string.Join(", ", deferred));
      Console.WriteLine(divider);


      // Exp. They are different because we singed
save to list so adding 5 will not change it

      // Individual assignment A
      Console.WriteLine($"\n Take 3 max numbers:
{string.Join(",",
deferred.OrderDescending().Take(3))}");

      // Exp. It's logical that first of all we
have to sort our list before
      // if we want to get the biggest numbers with
take afterwords


   }
}
```

```
namespace Lab12;


public class BuildingEntry
{
   public string Code { get; }
```

```
    public string Address { get; }
    public int FloorCount { get; }

    public BuildingEntry(string code, string address,
int floorCount)
    {
        Code = code;
        Address = address;
        FloorCount = floorCount;
    }



    public string GetInfo()
    {
        return $"Code: {this.Code}, address —
{this.Address}, floorCount: {this.FloorCount}";
    }

}
```

ШІ використовувався тільки для ресьорчу.